

Apache Airflow

What is Apache Airflow?

Apache Airflow is an open-source workflow orchestration platform used to:

- Schedule jobs
- Automate pipelines
- Manage task dependencies
- Monitor workflows

It was originally developed by Airbnb and later became an Apache Software Foundation project.

Airflow is widely used in:

- Data Engineering
- ETL/ELT Pipelines
- Machine Learning Workflows
- Cloud Automation
- Batch Processing

The core idea of Airflow is:

Define workflows as code using Python.

Workflow Orchestration

Workflow orchestration refers to:

- Managing task execution order
- Handling dependencies
- Scheduling workflows
- Monitoring execution
- Retrying failed tasks

Example workflow:

Collect Data → Clean Data → Train Model → Deploy Model

Instead of manually executing every step, Airflow automates the entire process.

DAG in Airflow

DAG – Directed Acyclic Graph

A DAG represents a workflow in Airflow.

Directed

Tasks execute in a defined order.

$A \rightarrow B \rightarrow C$

Acyclic

Loops are not allowed.

Invalid example:

$A \rightarrow B \rightarrow A$

Graph

Tasks and dependencies together form a graph structure.

Apache Airflow Architecture

High-Level Architecture

Web Server $\rightarrow$ Metadata Database $\leftarrow$ Scheduler $\rightarrow$ Executor $\rightarrow$ Workers

Components of Airflow

1. DAG Files

DAG files are Python scripts stored inside:

airflow/dags/

These files define:

- Workflow logic
 - Scheduling
 - Task dependencies
-

2. Scheduler

The Scheduler is the core component of Airflow.

Responsibilities:

- Reads DAG files
- Checks schedules
- Creates task instances
- Triggers workflows
- Manages dependencies

Example:

Run ETL pipeline every day at 2 AM

3. Metadata Database

Stores Airflow metadata such as:

- DAG information
- Task states
- Logs
- Execution history
- User information

Common databases:

- PostgreSQL
- MySQL

SQLite is not recommended for production.

4. Web Server

Provides the Airflow UI for monitoring and management.

Features:

- Monitor DAGs
- Trigger workflows manually
- Check logs
- Retry failed tasks
- View DAG graphs

Default URL:

<http://localhost:8080>

5. Executor

The Executor determines how and where tasks are executed.

Types of Executors

Executor	Description
SequentialExecutor	Executes one task at a time
LocalExecutor	Executes tasks in parallel on same machine
CeleryExecutor	Distributed execution using worker nodes
KubernetesExecutor	Executes tasks in Kubernetes pods

SequentialExecutor

Characteristics:

- Default executor
- Single task execution
- Mainly used for testing

Architecture:

Scheduler → Single Process Execution

LocalExecutor

Characteristics:

- Parallel execution
- Runs on same machine

Architecture:

Scheduler → Multiple Local Processes

CeleryExecutor

Characteristics:

- Distributed execution
- Multiple worker nodes
- Scalable

Uses:

- Celery
- Redis/RabbitMQ

Architecture:

Scheduler → Queue → Workers

KubernetesExecutor

Characteristics:

- Each task runs in separate Kubernetes pod
- Highly scalable
- Cloud-native

Architecture:

Scheduler → Kubernetes Pods

6. Workers

Workers execute the actual tasks.

Examples:

- Spark jobs
 - SQL queries
 - API calls
 - ML model training
 - File processing
-

Task Lifecycle in Airflow

No Status → Scheduled → Queued → Running → Success / Failed

Important Airflow Concepts

Operators

Operators define the action performed by a task.

Common Operators

Operator	Purpose
PythonOperator	Executes Python code
BashOperator	Executes shell commands
EmailOperator	Sends emails
SQLExecuteQueryOperator	Executes SQL queries
SparkSubmitOperator	Submits Spark jobs

Sensors

Sensors wait for specific events or conditions.

Examples:

- Waiting for a file
- Waiting for API response

Example:

Wait until CSV file appears in S3

Hooks

Hooks provide connections to external systems.

Examples:

- MySQLHook
- PostgresHook
- S3Hook

Used for:

- Database connectivity
 - Cloud storage access
-

XCom

XCom stands for Cross Communication.

Used to exchange data between tasks.

Example:

Task A sends data → Task B receives data

Scheduling in Airflow

Airflow supports:

- Cron scheduling
- Preset schedules
- Manual execution

Schedule	Meaning
@daily	Runs daily
@hourly	Runs hourly
@weekly	Runs weekly
0 2 * * *	Runs daily at 2 AM

Retries and Failure Handling

Airflow supports:

- Automatic retries
- Retry delays
- Email alerts
- SLA monitoring

Example:

```
retries=3,  
retry_delay=timedelta(minutes=5)
```

Airflow Logging

Every task generates logs for:

- Debugging
- Monitoring
- Auditing

Logs can be stored in:

- Local storage
- Amazon S3
- Azure Blob Storage

- Google Cloud Storage
-

Airflow with Data Engineering

Typical ETL pipeline:

Database → Spark → Data Warehouse → Dashboard

Airflow orchestrates the complete workflow.

Advantages of Apache Airflow

- Python-based workflows
 - Scalable architecture
 - Excellent monitoring UI
 - Strong dependency management
 - Fault tolerance and retries
 - Integration with cloud platforms and big data tools
 - Open-source and customizable
-

Limitations of Airflow

Limitation	Explanation
Not real-time	Mainly designed for batch processing
Complex setup	Production setup can be difficult
Heavy metadata usage	Requires optimization
DAG parsing overhead	Large DAGs may slow Scheduler

Airflow vs Cron

Feature	Cron	Airflow
---------	------	---------

Scheduling	Yes	Yes
Dependency Management	No	Yes
Monitoring UI	No	Yes
Retry Mechanism	Limited	Advanced
Scalability	Low	High
Workflow Visualization	No	Yes

Real-World Example

E-Commerce Pipeline

Extract Orders → Transform Data → Load into Snowflake → Generate Reports → Send Email

Airflow automates the complete workflow daily.
